

SkyPoints Global Airline Loyalty Data Engineering Platform

Senior Azure Data Engineer Architecture, Execution & Interview Master Guide

Scale Target: Multi-Billion Records/Day (~2-5 TB uncompressed)	Storage: ADLS Gen2 (stskypointsspeubmfodhieo)	Compute: Azure Databricks (dbw-skypoints-loyalty)
Orchestration: Azure Data Factory (adf-skypoints-loyalty)	Data Model: Snowflake DDL / Delta Country Marts	Verification: 100% Tests Passing & Deployed Live

1. Medallion Architecture & End-to-End Data Flow

SkyPoints processes millions to billions of member profiles and mileage redemption transactions daily across global airline partners. The system implements the **Medallion Architecture (Bronze → Silver → Gold)** on Azure to guarantee petabyte scalability, strict data quality, and compliance.

Layer	Container / Storage Path	Key Transformations & Business Logic	Output Schema / Marts
BRONZE (Landing / Raw)	ADLS Gen2 landing/bronze/	- Raw ingestion of member profiles (sample_member_feed.dat) & redemptions JSON. - Immutable audit log storing raw payloads, source file names, line numbers, and ingestion timestamps.	RAW_MEMBER_PROFILES RAW_REDEMPTION_FEED
SILVER (Staging / Cleansed)	ADLS Gen2 silver/(Delta Lake)	- Header filter ([H]) and Detail parser ([D]). Age derivation from DOB (accounting for leap year drift). Stale_Member flag (days_since_flight > 90 or never flown). - Zero-padding DOB recovery (3051985 → 03051985). - Dead-letter quarantine routing for invalid records.	STG_MEMBER_PROFILES quarantine_members
GOLD (Marts / Serving)	ADLS Gen2 gold/(Delta Lake)	Latest Record Wins window deduplication (ROW_NUMBER()). - Dynamic partition overwrite routing into country tables (table_usa, table_ind, table_can, table_phil, table_au). - Explode nested JSON redemptions array into fact_redemptions. Member 360 unified analytical view via Broadcast Hash Join.	table_can, table_ind fact_redemptions member_redemptions_360

2. Step-by-Step Pipeline Execution Guide

Execution Environment	Exact Commands to Run	Observed / Expected Outputs
1. Local Execution (Full ETL Pipeline)	python -m src.pipeline	Runs full Medallion pipeline. Ingests landing feeds, validates DQ, evaluates Age and Stale flags, performs latest-wins deduplication, routes to data/output/country_tables/, and syncs live to ADLS Gen2.
2. Automated Tests (100% Pass Rate)	python -m pytest -v	Executes 8 automated unit tests verifying header filtering, JSON explosion, DOB zero-padding recovery, age logic, flight-date sequence gates, and country routing. Result: 8 passed in 0.33s (100% pass).
3. Azure Data Factory (Cloud Orchestration)	az datafactory pipeline create-run \ --factory-name adf-skypoints-loyalty \ --resource-group rg-skypoints-loyalty \ --name pipeline_skypoints_orchestrator	Executes GetMetadata gatekeeper validation on ADLS Gen2 landing container. Verifies file presence and sizes, validates schemas, and signals downstream compute. Verified Live Run ID: eba929dc-b71a-11f1-ad17-ee2e98e90827 (Succeeded).
4. Azure Databricks (Serverless Compute)	1. Open Databricks workspace URL. 2. Navigate to Workspace → Users → email . 3. Import notebooks/skypoints_databricks_pipeline.py. 4. Select Serverless compute and click Run All .	Streams raw landing blobs from ADLS Gen2, builds PySpark DataFrames, performs window deduplication, and registers Delta Lake tables (stg_member_profiles, table_usa, table_ind, table_can, fact_redemptions) in Unity Catalog.

3. Detailed Component Deep-Dive & Code Breakdown

3.1 Ingestion, Header Stripping & Delimiter Alignment Trap

The Problem: The member flat file uses mixed records (|H|... header, |D|... details). Because lines begin with a leading pipe (e.g. |D|Elena|223457|...), a standard split produces an empty token at index 0.

PySpark Implementation:

```
# 1. Filter out Header (|H|) lines and blank lines
detail_df = raw_df.filter(F.col('value').startswith('|D|') | F.col('value').startswith('D|'))

# 2. Strip leading pipe with regex before splitting to prevent index offset
split_col = F.split(F.regexp_replace(F.col('value'), '^\\|', ''), '\\|')

# 3. Map tokens to strongly-typed columns
parsed_df = detail_df.select(
    F.trim(split_col.getItem(1)).alias('member_name'),
    F.trim(split_col.getItem(2)).alias('member_id'),
    F.to_date(F.trim(split_col.getItem(3)), 'yyyyMMdd').alias('enrollment_date'),
    F.to_date(F.trim(split_col.getItem(4)), 'yyyyMMdd').alias('last_flight_date'),
    F.nullif(F.trim(split_col.getItem(5)), '').alias('tier_code'),
    F.upper(F.nullif(F.trim(split_col.getItem(8)), '')).alias('country'),
    F.lpad(F.trim(split_col.getItem(9)), 8, '0').alias('raw_dob')
)
```

3.2 The DOB Leading Zero Recovery (Data Hygiene)

The Trap in Sample Data: In the assessment's intermediate table, DOB was displayed as 3051985 instead of 03051985. Upstream legacy systems frequently cast strings to integers, silently discarding leading zeroes for January through September births.

The Solution: We apply LPAD(raw_dob, 8, '0') to restore the leading zero, then coalesce MMdyyyy and ddMMyyyy formats:

F.coalesce(F.to_date(col('raw_dob'), 'MMdyyyy'), F.to_date(col('raw_dob'), 'ddMMyyyy')).

3.3 Staging Derived Columns (Deliverable 2: Age & Stale Member Flag)

1. Age Derivation: F.floor(F.datediff(F.current_date(), F.col('date_of_birth')) / 365.25).cast('int') handles leap year drift accurately across decades.

2. Stale_Member_Flag: A member is marked Stale ('Y') if: (a) They have never flown (last_flight_date IS NULL), or (b) Days since their last flight exceeds 90 days (datediff(current_date, last_flight_date) > 90). Otherwise marked 'N'.

3.4 'Latest Record Wins' Window Deduplication across Country Relocations (Deliverable 3)

The Relocation Problem: When members move to another country, they must not maintain active profiles across multiple country marts. In the assessment dataset, member **Elena ('223457')** initially had a flight in the USA (2012), but subsequently moved to Canada with a 2024 flight.

PySpark Window Ranking Logic:

```
window_spec = Window.partitionBy('member_id').orderBy(
    F.col('last_flight_date').desc_nulls_last(),
    F.col('enrollment_date').desc()
)
deduped_df = stg_df.withColumn('rank', F.row_number().over(window_spec)) \
    .filter(F.col('rank') == 1) \
    .drop('rank')

# Dynamic Delta Partition Overwrite by Country (Scales to billions of rows)
deduped_df.write.format('delta').mode('overwrite').partitionBy('country').saveAsTable('members_by_country')
```

3.5 Semi-Structured JSON Redemption Flattening & Member 360 (Deliverable 4)

Array Explosion: The partner feed provides a root member_id and a nested array redemptions: [{txn_id, txn_date, partner, miles_redeemed, status}]. Using F.explode(F.col('redemptions')) unpacks each transaction into an individual row in fact_redemptions.

Broadcast Hash Join: Aggregating redemptions produces a compact lookup table (total miles, completed miles, pending miles). By applying F.broadcast(redemption_agg_df), the smaller table is copied to all executor nodes, eliminating cluster-wide network shuffle when joining against billions of member profile records.

3.6 Data Quality Quarantine Dead-Letter Pattern (Deliverable 5)

In production petabyte-scale pipelines, encountering a corrupted record must not abort a multi-hour batch execution. Records failing quality gates (e.g. missing member_id, flight date preceding enrollment date, invalid date formats) are filtered into an append-only Delta table quarantine_members with explicit error tags (rejection_reason). The validated batch continues processing uninterrupted.

4. Data Modeling & DDL Specifications (Snowflake & Delta Lake)

The project provides complete production DDLs for both **Snowflake** (sql/01_ddl_snowflake.sql) and **Azure Databricks Delta Lake** (sql/02_ddl_azure_delta.sql).

```
-- Snowflake Staging DDL with Micro-Partition Clustering
CREATE OR REPLACE TABLE STAGING.STG_MEMBER_PROFILES (
  member_id          VARCHAR(18) NOT NULL,
  member_name        VARCHAR(255) NOT NULL,
  enrollment_date     DATE NOT NULL,
  last_flight_date    DATE,
  tier_code           VARCHAR(5),
  country            VARCHAR(10) NOT NULL,
  date_of_birth       DATE,
  age                NUMBER(3, 0),          -- Deliverable 2 Derived
  stale_member        VARCHAR(1) NOT NULL,  -- Deliverable 2 Derived
  CONSTRAINT pk_stg_member PRIMARY KEY (member_id)
) CLUSTER BY (country, member_id);

-- Canada Target Mart with SCD Tracking
CREATE OR REPLACE TABLE MARTS.TABLE_CAN (
  member_id          VARCHAR(18) NOT NULL,
  member_name        VARCHAR(255) NOT NULL,
  country            VARCHAR(10) DEFAULT 'CAN',
  effective_start_date  TIMESTAMP_NTZ NOT NULL,
  is_current          BOOLEAN DEFAULT TRUE,
  last_updated_at     TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP(),
  CONSTRAINT pk_table_can PRIMARY KEY (member_id)
) CLUSTER BY (member_id, last_flight_date);
```

5. Multi-Billion Scale Engineering Strategy

Processing billions of rows daily (~2-5 TB daily) demands specialized distributed engineering techniques to prevent out-of-memory errors, cluster timeouts, and excessive cloud spend:

Scaling Challenge	Architectural Solution	Technical Mechanism & Impact
File Discovery Bottleneck	Databricks Auto Loader (cloudFiles)	Uses Azure Event Grid blob creation notifications rather than listing directories. Discovers millions of files incrementally in sub-second time.
Multi-Country Writing Skew	Delta Dynamic Partition Overwrite	Writing with partitionBy('country') allows Spark's Catalyst engine to stream rows into partition subdirectories in a single shuffle pass, avoiding 50 individual table scan loops.
Small-File Degeneracy	Delta Compaction & Liquid Clustering	Executing OPTIMIZE members_by_country ZORDER BY (member_id) merges millions of small 10MB Parquet snippets into uniform 1GB files and co-locates data on disk.
Wide-Shuffle Memory Exhaustion	Broadcast Hash Joins (BHJ)	Broadcasting the aggregated redemptions table across worker nodes avoids shuffling the multi-terabyte member profile dataset across the cluster network.

6. One-Liner Technical Interview Master Cheat Sheet

Crisp, authoritative one-liner responses to technical questions for your senior interview:

Q1: What is the primary entity key for member profiles?

→ Member ID is the primary key; although the prompt table marked Member Name as key, in real loyalty programs names have collisions, requiring a unique business surrogate key.

Q2: How did you fix the dropped leading zero in the DOB column?

→ We used LPAD(raw_dob, 8, '0') to restore dropped leading zeroes (converting '3051985' to '03051985') before casting to ISO date.

Q3: What logic determines if a member is flagged as Stale?

→ A member is marked Stale ('Y') if their last flight date is NULL (never flown) or if datediff(current_date, last_flight_date) > 90 days.

Q4: How does the pipeline handle members moving between countries?

→ We apply window ranking ROW_NUMBER() partitioned by member_id and ordered by last_flight_date DESC; rank 1 routes to the new country table, and older country records are deactivated.

Q5: Why did you use ADLS Gen2 with Hierarchical Namespace (HNS)?

→ HNS enables true atomic directory renames and POSIX access control, essential for Delta Lake ACID transactions and petabyte-scale throughput.

Q6: How do you parse the semi-structured JSON redemption feed in PySpark?

→ We use PySpark's explode() function on the nested 'redemptions' array to unpack each transaction into a discrete, queryable row.

Q7: Why use a Broadcast Hash Join when joining Redemptions to Member Profiles?

→ The aggregated redemptions table is orders of magnitude smaller than the multi-billion member table; broadcasting avoids cluster-wide network shuffles.

Q8: Why not use standard Python/Pandas for this pipeline?

→ Billions of daily records represent 2-5 TB uncompressed; Pandas is single-threaded and memory-bound (OOM), whereas PySpark distributes computation across worker nodes.

Q9: What is the purpose of GetMetadata activities in Azure Data Factory?

→ They serve as cost-optimization gatekeepers, validating that daily landing files exist and are non-empty before spinning up expensive compute clusters.

Q10: What happens when a malformed record enters the pipeline?

→ It routes to an append-only dead-letter quarantine table (quarantine_members) with failure reason tags, preventing job abortion.

Q11: How do you prevent data skew when writing to country target tables?

→ We use Delta Dynamic Partition Overwrites partitioned by country, allowing Spark to write into target partition directories in a single distributed pass.

Q12: How do you mitigate the 'small-file problem' in Delta Lake?

→ We run OPTIMIZE members_by_country ZORDER BY (member_id) to compact small files into 1GB Parquet blocks and co-locate records on disk.

Q13: Why bypass spark.conf.set on Databricks Serverless compute?

→ Serverless enforces strict Unity Catalog security and blocks cluster-level Hadoop filesystem modifications; using the Azure Storage SDK provides native cloud stream access.

Q14: How is incremental processing handled at billions scale?

→ We use Databricks Auto Loader (cloudFiles) for event-driven file ingestion and Delta Lake MERGE INTO for atomic change data capture (CDC).

Q15: How did you handle the schema discrepancy regarding the Post Code column?

→ The markdown spec listed Post Code at position 9, but the sample flat file omitted it; our parser uses flexible token length mapping to prevent off-by-one errors.

Q16: How do you validate that flight dates are logical?

→ Our validation framework rejects records where last_flight_date < enrollment_date, routing them to the quarantine dead-letter sink.

Q17: What Delta Lake feature guarantees ACID transactions during simultaneous writes?

→ Delta Lake uses optimistic concurrency control backed by JSON transaction logs (_delta_log/) stored in ADLS Gen2.

Q18: How do you handle schema evolution if the partner airline adds a new field to the JSON feed?

→ In Databricks Auto Loader, we enable .option('cloudFiles.schemaEvolutionMode', 'addNewColumns') to capture new fields automatically into Silver Delta tables.

Q19: What is the difference between SCD Type 1 and Type 2 in the country target tables?

→ SCD Type 1 overwrites the prior country record directly; SCD Type 2 maintains history by adding is_current = FALSE and effective_end_date timestamps.

Q20: Why are raw payloads preserved verbatim in the Bronze layer?

→ Storing unparsed raw payloads guarantees an immutable audit trail, allowing downstream logic to be corrected and replayed if source systems send unexpected formatting.

Q21: What role does Azure Key Vault play in this architecture?

→ Azure Key Vault securely stores ADLS Gen2 access keys and service principal credentials, referenced dynamically by ADF Linked Services and Databricks secret scopes.

Q22: How does Snowflake micro-partitioning compare to Delta Lake partitioning?

→ Snowflake micro-partitions automatically without manual partition maintenance, while Delta Lake on ADLS Gen2 uses directory partitioning and Z-Ordering for columnar file layout.

Q23: How do you monitor pipeline health in Azure Data Factory?

→ ADF integrates with Azure Monitor and Log Analytics to track activity durations, trigger alerts on failure activities, and emit pipeline run metrics.

Q24: What is the effect of using `.desc_nulls_last()` in the deduplication window?

→ It ensures members with valid flight dates always take precedence over records where flight dates are null or missing.

Q25: Why is unity catalog important in Azure Databricks?

→ Unity Catalog provides centralized access governance, data lineage, automated auditing, and seamless cross-workspace sharing across ADLS Gen2 data assets.